



```
In [1]: from tensorflow.keras.datasets import cifar10
        (x_train, y_train), (x_test, y_test) = cifar10.load_data()
```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>
170498071/170498071 ————— 165s lus/step

```
In [2]: import numpy as np # for numerical operations
        import matplotlib.pyplot as plt

        from tensorflow.keras.datasets import cifar10 # CIFAR-10 dataset
        from tensorflow.keras.utils import to_categorical
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Drop
```

```
In [3]: # Load CIFAR-10 dataset
        (x_train, y_train), (x_test, y_test) = cifar10.load_data()
        print(x_train.shape, y_train.shape) # training data
        print(x_test.shape, y_test.shape)   # testing data
```

```
(50000, 32, 32, 3) (50000, 1)
(10000, 32, 32, 3) (10000, 1)
```

```
In [4]: # Normalize pixel values (0-255 → 0-1)
        x_train = x_train / 255.0
        x_test = x_test / 255.0

        # Convert labels to one-hot encoding
        y_train = to_categorical(y_train, 10)
        y_test = to_categorical(y_test, 10)
```

```
In [6]: from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Drop

        model = Sequential()

        # Input layer
        model.add(Input(shape=(32,32,3)))

        # First Convolution Block
        model.add(Conv2D(32,(3,3), activation='relu'))
        model.add(MaxPooling2D(pool_size=(2,2)))

        # Second Convolution Block
        model.add(Conv2D(64,(3,3), activation='relu'))
        model.add(MaxPooling2D(pool_size=(2,2)))

        # Third Convolution Block
        model.add(Conv2D(128,(3,3), activation='relu'))
        model.add(MaxPooling2D(pool_size=(2,2)))

        # Flatten
        model.add(Flatten())

        # Dense layers
```

```

model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(10, activation='softmax'))

```

```

In [7]: # Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

```

In [ ]: # Train model
history = model.fit(
    x_train, y_train,
    epochs=10, # number of iterations
    batch_size=64, # samples per batch
    validation_data=(x_test, y_test) # validation set
)

```

Epoch 1/10

782/782  **25s** 27ms/step - accuracy: 0.3721 - loss: 1.6838 - val_accuracy: 0.5164 - val_loss: 1.3247

Epoch 2/10

782/782  **40s** 52ms/step - accuracy: 0.5277 - loss: 1.3213 - val_accuracy: 0.5929 - val_loss: 1.1471

Epoch 3/10

782/782  **45s** 58ms/step - accuracy: 0.5902 - loss: 1.1660 - val_accuracy: 0.6289 - val_loss: 1.0538

Epoch 4/10

782/782  **42s** 54ms/step - accuracy: 0.6304 - loss: 1.0639 - val_accuracy: 0.6496 - val_loss: 0.9832

Epoch 5/10

782/782  **46s** 59ms/step - accuracy: 0.6619 - loss: 0.9730 - val_accuracy: 0.6662 - val_loss: 0.9406

Epoch 6/10

782/782  **43s** 55ms/step - accuracy: 0.6836 - loss: 0.9096 - val_accuracy: 0.6789 - val_loss: 0.8977

Epoch 7/10

782/782  **45s** 58ms/step - accuracy: 0.7014 - loss: 0.8615 - val_accuracy: 0.6837 - val_loss: 0.9091

Epoch 8/10

782/782  **42s** 53ms/step - accuracy: 0.7184 - loss: 0.8094 - val_accuracy: 0.6835 - val_loss: 0.9053

Epoch 9/10

782/782  **44s** 57ms/step - accuracy: 0.7341 - loss: 0.7662 - val_accuracy: 0.7062 - val_loss: 0.8590

Epoch 10/10

696/782  **4s** 52ms/step - accuracy: 0.7517 - loss: 0.7169

```

In [ ]: # Evaluate model on test data

```

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)

print("Test Accuracy:", test_accuracy)
```

```
In [ ]: # Plot training vs validation accuracy
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```